Práctica 4. Entrenamiento, Evaluación y Despliegue.

Con el modelo construido y los datos listos, es hora de entrenarlo, un proceso computacionalmente intensivo. Luego, lo evaluarán y lo desplegarán como un servicio web, haciéndolo accesible a otras aplicaciones.

Objetivos de Aprendizaje:

- Entrenar un modelo de aprendizaje profundo en la nube.
- Evaluar el rendimiento de un modelo generativo.
- Empaquetar la IA en un contenedor (Docker).
- Desplegar el modelo como una API REST en una plataforma de IA.
- Considerar los aspectos de seguridad del modelo desplegado.

Contenidos del Módulo Cubiertos:

- `Desarrollo de aplicaciones de IA`: Plataformas de IA (Azure Machine Learning, AWS SageMaker), evaluación de resultados.
- `Evaluación de la Convergencia tecnológica`: Cloud, Seguridad en la convergencia tecnológica.

Pasos a Realizar:

1. **Entrenamiento en la nube.**
    - **Configuración:** Configurar un entorno de entrenamiento en la nube (e.g., Google Colab con GPU, una instancia de AWS EC2 con GPU, o usando servicios como Azure ML/SageMaker).
    - **Entrenamiento:** Escribir el script que carga los datos desde el bucket de almacenamiento, instancia el modelo y comienza el entrenamiento (`model.fit()`). Es fundamental guardar "checkpoints" del modelo periódicamente.
    - **Monitorización:** Observar la evolución de la pérdida (loss) durante el entrenamiento para detectar si el modelo está aprendiendo correctamente. El entrenamiento puede durar varias horas.

2. **Evaluación y generación de texto.**
    - **Evaluación Cuantitativa:** Medir la **perplejidad** del modelo sobre un conjunto de datos de prueba. Investigar qué es la perplejidad y cómo se interpreta.
    - **Evaluación Cualitativa:** Crear una función de generación de texto que utilice el modelo entrenado. Partiendo de una frase inicial ("seed"), el modelo predecirá la siguiente palabra, se añadirá a la secuencia, y el proceso se repetirá.
    - **Experimentación:** Probar diferentes estrategias de generación (e.g., greedy search vs. sampling con temperatura) y analizar la calidad del texto generado.

3. **Contenerización y API.**
    - **API con Flask/FastAPI:** Envolver el modelo entrenado en una API REST simple. Debe tener un endpoint (e.g., `/generate`) que acepte un texto inicial y devuelva el texto generado por la IA.
    - **Docker:** Escribir un `Dockerfile` para empaquetar la aplicación Flask y todas sus dependencias (TensorFlow, etc.) en una imagen de contenedor. Esto asegura que el modelo se ejecute de la misma manera en cualquier lugar.
    - **Pruebas Locales:** Construir y ejecutar el contenedor Docker localmente para verificar que la API funciona correctamente.

4. **Despliegue en la nube.**
    - **Investigación:** Comparar diferentes opciones de despliegue: IaaS (máquina virtual), PaaS (Heroku) y servicios específicos de IA (Azure Container Apps, AWS App Runner, SageMaker Endpoints).
    - **Despliegue:** Desplegar el contenedor en la plataforma elegida. El objetivo es tener una URL pública a la que se pueda llamar para obtener texto generado por la IA.
    - **Seguridad:** Investigar y documentar al menos dos medidas de seguridad para la API (e.g., autenticación con clave de API, limitación de tasa de peticiones) para prevenir abusos.